

Curso desarrollo de aplicaciones con tecnologías web

UF1843. Aplicación de técnicas de usabilidad y accesibilidad en el entorno cliente.

Accesibilidad web

Definición de accesibilidad web

Ventajas y dificultades en la implantación de la accesibilidad web

Normativa y estándares sobre accesibilidad web

Guías para el cumplimiento de normativas y estándares

Descripción del proceso de la conformidad en accesibilidad web

Tecnologías donde la accesibilidad es aplicable

Herramientas para la validación de la accesibilidad

Evolución de la accesibilidad. Nuevas tendencias

Definición de accesibilidad web

La accesibilidad web es la práctica de diseñar y desarrollar sitios web para que puedan ser utilizados por la mayor cantidad de personas posible, incluidas aquellas con discapacidades visuales, auditivas, motoras o cognitivas.

Ejemplo:

- Un sitio accesible debe permitir la navegación mediante teclado sin necesidad de usar el ratón.
- Las imágenes deben incluir un atributo alt que describa su contenido para personas con discapacidad visual.

Ventajas y dificultades en la implantación de la accesibilidad web

Ventajas:

- Permite que más usuarios accedan al contenido.
- Mejora la usabilidad para todos, no solo para personas con discapacidad.
- Aumenta el posicionamiento en buscadores (SEO).
- Puede ser obligatorio según la legislación en algunos países.

Dificultades:

- Puede requerir cambios en el código y diseño.
- No todas las tecnologías son compatibles con las pautas de accesibilidad.
- Requiere pruebas y validaciones adicionales.

Normativa y estándares sobre accesibilidad web

Las normas más importantes sobre accesibilidad son:

- **WCAG (Web Content Accessibility Guidelines):** Directrices internacionales desarrolladas por el W3C.
- **Ley de Accesibilidad Europea:** Exige accesibilidad en sitios web de organismos públicos y privados en algunos países.
- **ADA (Americans with Disabilities Act):** Norma de accesibilidad en EE.UU.
- **EN 301 549:** Normativa europea sobre accesibilidad en TIC.

Ejemplo de código accesible según WCAG:

```
<a href="documento.pdf" aria-label="Descargar el documento en formato PDF">Descargar documento</a>
```

El atributo `aria-label` ayuda a lectores de pantalla a interpretar correctamente el enlace. Sirve para proporcionar un nombre accesible a elementos interactivos (botones, enlaces, iconos, etc.) cuando no tienen texto visible o cuando su significado no es claro.

¿Qué hace `aria-label`?

1. Lectores de pantalla

- Sí, los lectores de pantalla lo leen en voz alta para describir el propósito del elemento.
- Si un botón solo tiene un icono visual (`<i class="bi bi-x"></i>`), el lector de pantalla no sabrá qué hace. Con `aria-label="Cerrar"`, el usuario con discapacidad auditiva escuchará “Cerrar”.

2. No afecta el SEO directamente

- No mejora el posicionamiento en Google como lo haría un texto visible o un atributo `alt` en imágenes.
- Pero mejora la accesibilidad, lo cual es un factor que Google valora para la experiencia de usuario.

3. No afecta el diseño ni la visibilidad

- A diferencia del atributo alt en imágenes, aria-label no se muestra en pantalla, solo lo perciben tecnologías de asistencia.

Ejemplo con un botón de cerrar sin texto visible

- **Sin aria-label (NO accesible)**

```
<button>  
  <i class="bi bi-x-circle"></i>  
</button>
```

Problema:

- Un usuario sin visión no sabe qué hace este botón.
- El lector de pantalla solo dirá “botón” sin más contexto.

- **Con aria-label (accesible)**

```
<button aria-label="Cerrar ventana">  
  <i class="bi bi-x-circle"></i>  
</button>
```

El lector de pantalla dirá: “Cerrar ventana”.

¿Cuándo usar aria-label?

- Cuando un elemento no tiene texto visible, pero debe ser entendido por lectores de pantalla.
- Para mejorar la accesibilidad en botones con solo iconos.
- Para elementos visuales sin un alt claro, como enlaces de descarga.

No es necesario si el texto visible ya describe correctamente la función del elemento.

Organismos regulatorios de la accesibilidad web

- **W3C (World Wide Web Consortium):** Define estándares como las WCAG.
- **Fundación ONCE:** Promueve la accesibilidad digital en España.
- **Section 508:** Regulación de accesibilidad en EE.UU.

Comparativa de normas y estándares

Normativa	Ámbito	Principales requisitos
WCAG 2.1	Internacional	Textos alternativos, navegación con teclado, contraste adecuado
Ley de Accesibilidad Europea	Europa	Aplicable a servicios digitales de empresas y gobierno
Section 508	EE.UU.	Exige accesibilidad en webs de organismos públicos
EN 301 549	UE	Accesibilidad en tecnologías de la información

Guías para el cumplimiento de normativas y estándares

Algunas pautas esenciales incluyen:

- Usar etiquetas semánticas en HTML (header, main, article, footer, nav, strong...).
- Garantizar suficiente contraste entre el texto y el fondo.
- Evitar contenido solo accesible con el ratón (asegurar navegación con teclado).
- Añadir subtítulos a los vídeos para personas con discapacidad auditiva.

Descripción de las pautas principales

Imágenes:

- Incluir siempre un atributo alt con una descripción clara.

Enlaces:

- Deben tener un texto claro que describa el destino (evitar “Haz clic aquí”).

Vídeo:

- Debe incluir subtítulos y transcripción del audio.

Ejemplo de HTML accesible:

```

```

Pautas para una navegación accesible

- Uso de encabezados estructurados con h1, h2, h3 etc.
- Evitar bloques de texto largos sin separación.
- Compatibilidad con lectores de pantalla.
- Evitar contenido que parpadee o se mueva demasiado rápido.

Ejemplo de etiquetas semánticas bien usadas:

```
<header>
  <h1>Mi Blog de Tecnología</h1>
  <nav>
    <ul>
      <li><a href="#inicio">Inicio</a></li>
      <li><a href="#articulos">Artículos</a></li>
      <li><a href="#contacto">Contacto</a></li>
    </ul>
  </nav>
</header>

<main>
  <article>
    <h2>Cómo mejorar la accesibilidad en tu web</h2>
    <p>La accesibilidad web es fundamental para garantizar que todos los usuarios puedan interactuar con el contenido.</p>
  </article>
</main>

<footer>
  <p>© 2025 Mi Blog de Tecnología</p>
</footer>
```

Encabezados estructurados correctamente:

```
<h1>Blog de Ciencia</h1>  
<h2>Artículos recientes</h2>  
<h3>Cómo funciona la energía solar</h3>  
<h3>Avances en inteligencia artificial</h3>
```

Buenas prácticas:

- Usar solo un <h1> por página (título principal).
- No saltar niveles (h2 → h4 sin h3).
- Evitar usar encabezados solo por tamaño visual, usa CSS para estilos.

Descripción del proceso de la conformidad en accesibilidad web

Para cumplir con las WCAG, los sitios deben cumplir con los niveles de conformidad:

- **Nivel A:** Requisitos básicos (navegación por teclado, etiquetas alt).
- **Nivel AA:** Mejoras en la accesibilidad (contraste adecuado, compatibilidad con tecnología asistida).
- **Nivel AAA:** Nivel más alto (narración de contenido, personalización avanzada).

Tecnologías donde la accesibilidad es aplicable

Tecnología	Aspectos de accesibilidad
HTML	Uso de etiquetas semánticas (header, main, article, footer...)
CSS	Uso de colores con buen contraste, evitar imágenes de fondo con información esencial
JavaScript	Evitar funciones que dependan solo del ratón
PDF	Permitir que los lectores de pantalla interpreten el contenido
XML/XSL	Estructurar datos de forma legible para herramientas de accesibilidad
Multimedia	Añadir subtítulos, transcripciones y audiodescripción

Ejemplo de accesibilidad en JavaScript:

```
document.getElementById("boton").addEventListener("keydown", function (event) {  
    if (event.key === "Enter") {  
        alert("Has pulsado Enter sobre el botón.");  
    }  
});
```

Este código permite activar el botón con la tecla Enter, mejorando la accesibilidad para usuarios con discapacidad motora.

Accesibilidad con teclado y tabindex

La navegación con teclado permite a los usuarios moverse entre los elementos interactivos de una web usando la tecla Tab.

Ejemplo básico de navegación con teclado:

```
<a href="pagina.html" tabindex="1">Enlace prioritario</a>
<button tabindex="2">Botón importante</button>
<input type="text" placeholder="Escribe aquí" tabindex="3">
```

tabindex="0": Sigue el orden natural de navegación con Tab.

tabindex="-1": Hace que un elemento no sea accesible con Tab, pero sí con JavaScript (focus()).

- **Ejemplo de salto de navegación:**

```
<nav>
  <a href="#main-content" class="skip-link">Saltar al contenido</a>
</nav>

<main id="main-content">
  <h2>Contenido principal</h2>
  <p>Este es el contenido principal de la página.</p>
</main>
```

Consejo: Usar un enlace de salto al contenido al inicio de la página mejora la experiencia de usuarios con lectores de pantalla.

- **Ejemplo de JavaScript para mejorar accesibilidad con teclado:**

js:

```
document.addEventListener("keydown", function (event) {  
    if (event.key === "Tab") {  
        console.log("El usuario está navegando con el teclado.");  
    }  
});
```

- **Ejemplo avanzado combinando accesibilidad y teclado:**

Aquí un botón solo activable con la tecla Enter para mejorar la experiencia en personas con movilidad reducida.

js;

```
document.getElementById("boton-accesible").addEventListener("keydown", function (event) {  
    if (event.key === "Enter") {  
        alert("Botón activado con el teclado");  
    }  
});
```

Esto permite que los usuarios puedan activar el botón sin necesidad de un ratón.

Herramientas para la validación de la accesibilidad

Existen diversas herramientas que permiten comprobar si una web cumple con las pautas de accesibilidad:

- **Basadas en navegador:**

- Lighthouse (en Chrome DevTools).
- WAVE (Web Accessibility Evaluation Tool).

- **Aplicaciones de escritorio:**

- NVDA (lector de pantalla para Windows).
- JAWS (lector de pantalla avanzado).

- **Servicios web externos:**

- Accessibility Checker (AChecker).
- axe DevTools.

Evolución de la accesibilidad. Nuevas tendencias

- **Inteligencia artificial:** Herramientas como AI-captioning pueden generar subtítulos automáticos en tiempo real.
- **Accesibilidad móvil:** Garantizar la compatibilidad con lectores de pantalla en dispositivos móviles.
- **Interfaces de voz:** Los asistentes como Siri y Alexa facilitan el acceso a la web para personas con discapacidad visual.
- **Dark Mode y personalización:** Algunos sitios permiten cambiar los colores para mejorar la visibilidad.

Ejercicio práctico: Validación de accesibilidad

Objetivo: Evaluar la accesibilidad de una página web con herramientas automatizadas.

1. Usa Lighthouse en Chrome para analizar la accesibilidad de una web.
2. Identifica errores comunes (bajo contraste, etiquetas alt faltantes, etc.).
3. Propón mejoras basadas en los resultados.

Anexos

Tabla tecnologías donde la accesibilidad es aplicable

Tecnología	Aplicaciones en Accesibilidad
HTML (Marcado accesible)	✓ Uso de etiquetas semánticas (<header>, <nav>, <main>, <footer>, <article>, etc.) para mejorar la estructura del contenido.
	✓ Uso de encabezados jerárquicos (<h1> a <h6>) para facilitar la navegación con lectores de pantalla.
	✓ Uso del atributo alt en imágenes para describir su contenido a usuarios con discapacidad visual.
	✓ Uso del atributo aria-label en botones o enlaces sin texto visible para que los lectores de pantalla puedan identificarlos.
	✓ Uso del atributo title en enlaces para proporcionar información adicional sobre su destino.
	✓ Uso de tabindex para mejorar la navegación con teclado, permitiendo definir el orden de tabulación.
	✓ Uso de role para definir el propósito de un elemento en la página y facilitar su interpretación por tecnologías asistivas.

Tecnología	Aplicaciones en Accesibilidad
CSS (Estilos accesibles)	 Uso de buen contraste entre el texto y el fondo para mejorar la legibilidad (mínimo 4.5:1).
	 Uso de focus-visible en los estilos para resaltar elementos interactivos sin afectar a usuarios que navegan con el ratón.
	 Uso de prefers-reduced-motion en @media para desactivar animaciones en usuarios que lo necesiten.
JavaScript (Interactividad accesible)	 Manejo de eventos de teclado (keydown, keyup) para permitir la navegación sin ratón.
	 Modificación dinámica de atributos aria (aria-expanded, aria-hidden, etc.) para mejorar la accesibilidad en elementos interactivos.
	 Implementación de alertas accesibles con role="alert" para notificaciones importantes.
SVG (Gráficos escalables)	 Uso de etiquetas title y desc en elementos SVG para proporcionar descripciones accesibles en gráficos y diagramas.
WAI-ARIA (Mejoras en accesibilidad)	 Uso de aria-live para actualizar contenido dinámico sin interrumpir la navegación.

Tecnología	Aplicaciones en Accesibilidad
	 Uso de aria-hidden="true" en elementos decorativos para evitar distracciones innecesarias en lectores de pantalla.
	 Uso de aria-describedby para asociar descripciones a elementos interactivos.

Tabla de etiquetas semánticas en HTML y su significado para los lectores de pantalla

Etiqueta	Función / Uso	Impacto en accesibilidad
Estructura y contenido		
<header>	Encabezado de la página o una sección.	Identificado como “Encabezado”.
<nav>	Contenedor de navegación con enlaces internos o externos.	Los lectores de pantalla lo anuncian como “Navegación”.
<main>	Contenido principal de la página.	Permite saltar directamente al contenido principal.
<section>	Agrupar contenido temáticamente.	Divide lógicamente la página en partes.
<article>	Contenido independiente, como blogs o noticias.	Se puede anunciar como un contenido autónomo.
<aside>	Contenido complementario, como barras laterales o widgets.	Se indica como información secundaria.
<footer>	Pie de página con información extra.	Los lectores de pantalla lo anuncian como “Pie de página”.

Etiqueta	Función / Uso	Impacto en accesibilidad
<figure>	Agrupar elementos gráficos como imágenes o videos.	Facilita la accesibilidad de elementos visuales.
<figcaption>	Proporciona una descripción accesible dentro de <figure>.	Se lee como texto adicional para la imagen o gráfico.
<time>	Representa una fecha u hora legible por máquinas.	Útil para SEO y accesibilidad.
<mark>	Resalta texto importante visualmente.	Puede ser leído como “Texto resaltado”.
<blockquote>	Cita en bloque de otro contenido.	Puede anunciarse como “Cita” en lectores de pantalla.
<q>	Representa una cita breve en línea.	Se interpreta como “Cita”.
<address>	Información de contacto, como dirección o teléfono.	Se trata como un bloque de datos de contacto.
<summary>	Título accesible dentro de un <details>.	Se usa para describir información oculta/ mostrada.
<details>	Contenedor desplegable interactivo.	Permite expandir o colapsar contenido.
<dialog>	Representa un cuadro de diálogo o ventana emergente (modal).	Se usa con aria-labelledby para accesibilidad.
<fieldset>	Agrupar elementos de formulario relacionados.	Mejora la organización en formularios.
<legend>	Proporciona una descripción accesible dentro de un <fieldset>.	Se usa para describir campos de formulario.
<datalist>	Proporciona opciones predefinidas en un <input>.	Facilita la selección de valores en formularios.

Etiqueta	Función / Uso	Impacto en accesibilidad
<output>	Muestra el resultado de cálculos o scripts interactivos.	Indica datos generados dinámicamente.
<progress>	Representa el progreso de una tarea, como una barra de carga.	Puede anunciarse como “Progreso de tarea”.
<meter>	Representa un valor dentro de un rango conocido.	Se usa para visualizaciones como una batería o puntuación.
Formato de texto (Negritas, Cursivas, etc.)		
	Texto en negrita con énfasis.	Se anuncia con énfasis adicional. Útil para destacar información importante.
	Texto en negrita sin énfasis semántico.	No tiene significado especial para accesibilidad. Solo es visual.
	Texto en cursiva con énfasis.	Se lee con énfasis adicional en lectores de pantalla.
<i>	Texto en cursiva sin énfasis semántico.	No tiene significado especial, solo estilización.
<u>	Texto subrayado.	No tiene valor semántico, pero se usa para énfasis visual.
<s>	Texto tachado.	Indica que algo ya no es válido o relevante.
<small>	Texto más pequeño.	Se usa para información secundaria o aclaraciones.
<abbr>	Abreviatura o acrónimo con un significado completo en title.	Se puede leer el significado completo si el lector lo permite.
<cite>	Indica el nombre de una obra citada.	Se usa para identificar fuentes de información.

Etiqueta	Función / Uso	Impacto en accesibilidad
<code>	Texto en formato de código de programación.	Puede anunciarse como “Bloque de código”.
<pre>	Bloque de código preformateado con espaciado exacto.	Mantiene la estructura del código, útil para accesibilidad.
<kbd>	Representa una entrada de teclado.	Se usa para mostrar combinaciones de teclas, útil para documentación.

Anexo: Atributos ARIA en Accesibilidad Web

¿Qué es ARIA?

ARIA (Accessible Rich Internet Applications) es un conjunto de atributos que mejoran la accesibilidad de las aplicaciones web, especialmente para usuarios que usan lectores de pantalla o navegan solo con el teclado.

Se utilizan para dar información adicional a los elementos HTML cuando los atributos nativos no son suficientes.

Tabla de Atributos ARIA Más Usados

Atributo	Descripción	Ejemplo
aria-label	Proporciona un nombre accesible a un elemento que no tiene texto visible.	<code><button aria-label="Enviar formulario"></button></code>
aria-labelledby	Asocia un elemento con otro que contiene su descripción accesible.	<code><div id="info">Información importante</div> <button aria-labelledby="info">Leer más</button></code>
aria-describedby	Relaciona un elemento con otro que da más detalles sobre su función o contenido.	<code><input type="text" aria-describedby="ayuda"> <small id="ayuda">Escribe tu nombre completo.</small></code>
aria-hidden	Indica si un elemento debe ser ignorado por los lectores de pantalla (true o false).	<code><div aria-hidden="true">Solo decorativo</div></code>

Atributo	Descripción	Ejemplo
aria-live	Define cómo los lectores de pantalla anuncian cambios en el contenido (polite, assertive).	<code><div aria-live="polite">Cargando...</div></code>
aria-expanded	Indica si un elemento desplegable (menú, acordeón) está abierto o cerrado.	<code><button aria-expanded="false">Menú</button></code>
aria-checked	Especifica si un checkbox o un radio está marcado (true, false, mixed).	<code><input type="checkbox" aria-checked="true"></code>
aria-disabled	Indica si un elemento está deshabilitado (diferente de disabled, ya que este último es nativo de HTML).	<code><button aria-disabled="true"> No disponible</button></code>
aria-pressed	Define si un botón tipo interruptor está activado o no (true o false).	<code><button aria-pressed="false"> Activar</button></code>
aria-current	Indica la página o sección actual en una navegación.	<code>Inicio</code>
aria-haspopup	Especifica si un botón abre un menú, un diálogo u otro tipo de contenido emergente.	<code><button aria-haspopup="menu"> Opciones</button></code>
aria-roledescription	Proporciona un nombre personalizado para el rol de un elemento.	<code><div role="alert" aria-roledescription="Mensaje de advertencia">¡Cuidado!</div></code>

Ejemplo Completo de Uso de ARIA

```
<nav>
  <ul>
    <li><a href="home.html" aria-current="page">Inicio</a></li>
    <li><a href="about.html">Sobre nosotros</a></li>
    <li>
      <button aria-expanded="false" aria-haspopup="menu" id="menu-btn">
        Menú ▼
      </button>
      <ul id="submenu" aria-hidden="true">
        <li><a href="services.html">Servicios</a></li>
        <li><a href="contact.html">Contacto</a></li>
      </ul>
    </li>
  </ul>
</nav>
```

Explicación:

- El enlace de Inicio tiene `aria-current="page"` porque es la página en la que estamos.
- El botón del menú usa `aria-expanded="false"` para indicar que el menú está cerrado.
- El `aria-haspopup="menu"` informa que el botón abre un menú emergente.
- `aria-hidden="true"` en el menú oculta el contenido para los lectores de pantalla hasta que sea visible.

Documentación oficial de ARIA está en la web del W3C (World Wide Web Consortium).

- Referencia oficial de ARIA en W3C: <https://www.w3.org/TR/wai-aria-1.2/>
- ARIA Authoring Practices Guide (APG): <https://www.w3.org/WAI/ARIA/apg/>
- Lista completa de atributos ARIA en MDN (Mozilla Developer Network):
<https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes>

En estas páginas puedes encontrar todos los atributos ARIA, ejemplos de uso y mejores prácticas.